<u>Experiment 2</u>

- ***<u>Aim</u>*** **–**
  To study and perform arithmetic operations on images:
  1. Addition Operation
  2. Subtraction Operation
  3. Multiplication Operation
  4. Division Operation

- ***<u>Tools & Tech-stack Used</u>*** **–**
  Google colab, Python, Numpy, Matplot, cv2.

- ***<u>Theory</u>*** **–**
  A digital image is represented as a matrix of pixel intensity values.
  For grayscale images, each pixel has one value (0–255).
  For color images, each pixel has three values (R, G, B).

  Image arithmetic operations perform mathematical calculations on corresponding pixel values of two images (or an image and a constant) to produce a new image.

  These operations are widely used in:

  - ➢ Image enhancement
  - ➢ Image blending
  - ➢ Change detection
  - ➢ Noise reduction

  For two images A and B:

  $$\mathbf{C(x,y) = A(x,y)\ op\ B(x,y);\ \{where\ op\ is\ +, -, \times, or\ \div\}}$$

  1. <u>*Image Addition*</u>: Pixel values of two images are added; C = A + B

     ➔ Produces a brighter image
     ➔ Used for image blending

2. *Image Subtraction*: Subtracts pixel values of one image from another; C = A−B

➔ Highlights differences between images
➔ Used in motion detection & background removal

3. *Image Multiplication*: Multiplies pixel values; C = A × B

➔ Darkens the image
➔ Used in masking and filtering

4. *Image Division*: Divides pixel values; C = A / B

➔ Used for illumination correction
➔ Helps normalize brightness

- ***Code*** –

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow

img_baboon = cv2.imread("baboon.png")
img_cameraman = cv2.imread("cameraman.png")
cv2_imshow(img_baboon)
cv2_imshow(img_cameraman)
```

```
cv2 reads the channels RGB backwards i.e BGR and matplot
reads them as RGB, so in matplot, reddominating becomes
blue and vice versa.
```

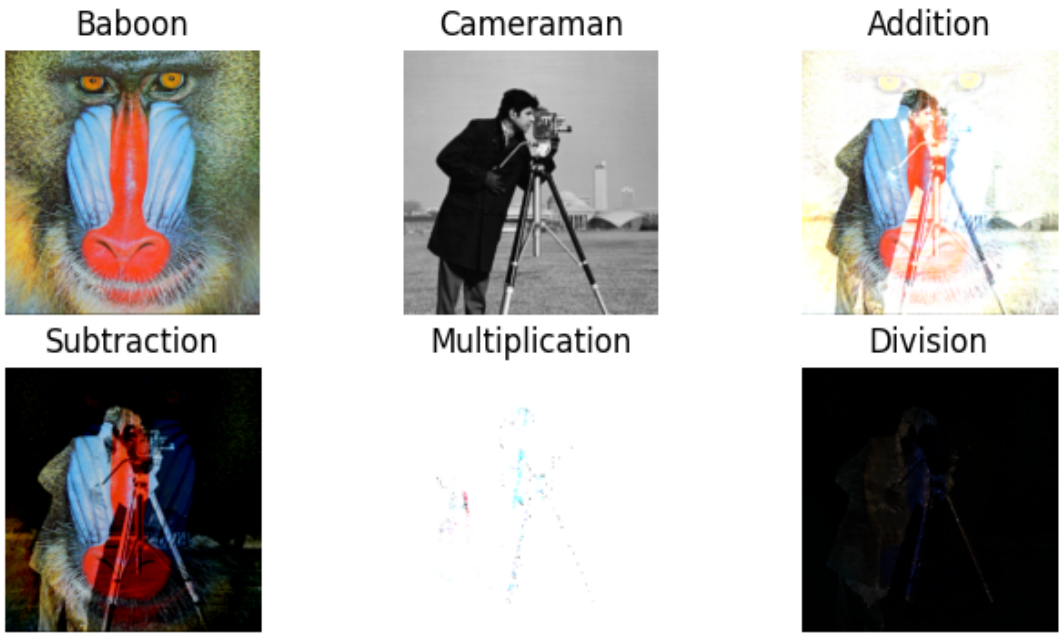
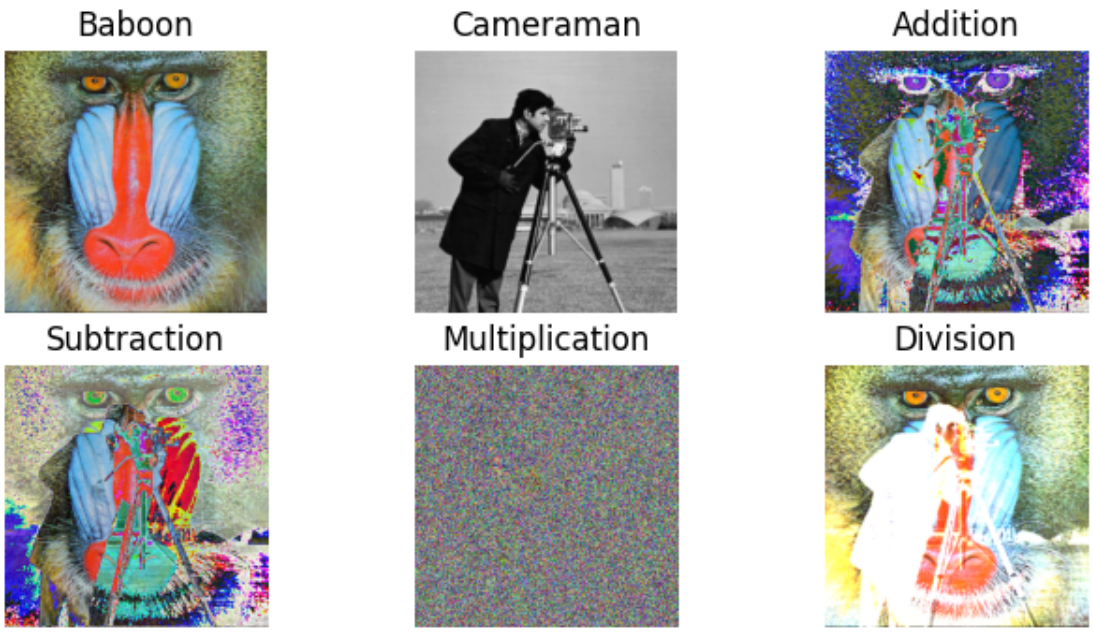
```
# Using cv2 method
img_add = cv2.add(img_baboon, img_cameraman)
img_sub = cv2.subtract(img_baboon, img_cameraman)
img_mul = cv2.multiply(img_baboon, img_cameraman)
img_div = cv2.divide(img_baboon, img_cameraman)


plt.figure(figsize=(8, 4))
```

```
plt.subplot(2, 3, 1)
plt.imshow(img_baboon[:, :, ::-1])
plt.title('Baboon')
plt.axis('off')

plt.subplot(2, 3, 2)
plt.imshow(img_cameraman)
plt.title('Cameraman')
plt.axis('off')

plt.subplot(2, 3, 3)
plt.imshow(img_add[:, :, ::-1])
plt.title('Addition')
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(img_sub[:, :, ::-1])
plt.title('Subtraction')
plt.axis('off')

plt.subplot(2, 3, 5)
plt.imshow(img_mul)
plt.title('Multiplication')
plt.axis('off')

plt.subplot(2, 3, 6)
plt.imshow(img_div)
plt.title('Division')
plt.axis('off')

# Using numpy method
img_addn = np.add(img_baboon, img_cameraman)
img_subn = np.subtract(img_baboon, img_cameraman)
img_mult = np.multiply(img_baboon, img_cameraman)
img_div = np.divide(img_baboon, img_cameraman)

plt.figure(figsize=(8, 4))
```

```
plt.subplot(2, 3, 1)
plt.imshow(img_baboon[:, :, ::-1])
plt.title('Baboon')
plt.axis('off')

plt.subplot(2, 3, 2)
plt.imshow(img_cameraman)
plt.title('Cameraman')
plt.axis('off')

plt.subplot(2, 3, 3)
plt.imshow(img_addn[:, :, ::-1])
plt.title('Addition')
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(img_subn[:, :, ::-1])
plt.title('Subtraction')
plt.axis('off')

plt.subplot(2, 3, 5)
plt.imshow(img_mult)
plt.title('Multiplication')
plt.axis('off')

plt.subplot(2, 3, 6)
plt.imshow(img_div[:, :, ::-1])
plt.title('Division')
plt.axis('off')
```

- ***Result*** –
  The arithmetic operations on images were successfully performed using addition, subtraction, multiplication, and division. Each operation produced a distinct visual effect on the output image.

➢ *Using cv2:*



➢ *Using numpy:*